

Evaluación Sumativa

Laboratorio Evaluado: Implementación de un Asistente RAG

Asignatura: IA Embebida en Sistemas Computacionales

Carrera: Ingeniería Informática

Modalidad: Equipos de 2 a 3 estudiantes

Presentación: Clase siguiente (5 a 7 minutos por equipo)

1. Contexto

Los modelos de lenguaje (LLM) son capaces de responder preguntas complejas, pero presentan una limitación importante: su conocimiento depende de la información disponible durante su entrenamiento.

Para resolver este problema, las organizaciones modernas utilizan arquitecturas **RAG (Retrieval Augmented Generation)**, las cuales permiten recuperar información desde documentos propios antes de generar una respuesta.

En esta evaluación deberán implementar un asistente basado en RAG capaz de responder preguntas utilizando documentos específicos del dominio seleccionado.

El objetivo no es construir una aplicación compleja, sino demostrar comprensión práctica de los siguientes conceptos:

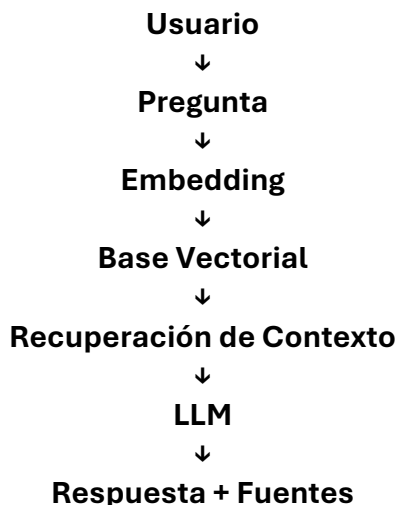
- Embeddings.
 - Búsqueda semántica.
 - Bases vectoriales.
 - Recuperación de contexto.
 - Integración con modelos de lenguaje.
-

2. Objetivo de Aprendizaje

Implementar una solución funcional basada en RAG capaz de recuperar información relevante desde un conjunto de documentos y utilizar dicha información para generar respuestas fundamentadas mediante un modelo de lenguaje.

3. Arquitectura Base

Todos los equipos deberán implementar la siguiente arquitectura:



No se evaluará el diseño arquitectónico, ya que esta arquitectura será común para todos los equipos.

La evaluación se centrará en la correcta implementación y funcionamiento de cada componente.

4. Casos de Uso Disponibles

Cada equipo deberá seleccionar **uno** de los siguientes escenarios.

Caso 1: Asistente Académico Universitario

Descripción

La universidad desea implementar un asistente capaz de responder consultas frecuentes de estudiantes utilizando documentación institucional.

Corpus documental sugerido

- Reglamento de asistencia.
- Reglamento de evaluaciones.
- Reglamento de prácticas.
- Reglamento de titulación.
- Calendario académico.

Preguntas de ejemplo

- ¿Cuál es el porcentaje mínimo de asistencia?
- ¿Qué ocurre si repruebo una asignatura?
- ¿Cuáles son los requisitos para titularme?
- ¿Cuándo finaliza el semestre?

Resultado esperado

El sistema debe responder utilizando únicamente información recuperada desde los documentos institucionales.

Caso 2: Asistente de Soporte Técnico

Descripción

Una empresa tecnológica desea implementar un asistente capaz de ayudar a usuarios y técnicos utilizando documentación interna.

Corpus documental sugerido

- Manual Docker.
- Manual Linux.
- Manual Git.
- Procedimientos de soporte.
- Preguntas frecuentes.

Preguntas de ejemplo

- ¿Cómo crear una imagen Docker?
- ¿Cómo actualizar un repositorio Git?
- ¿Cómo visualizar procesos en Linux?
- ¿Cómo abrir un ticket de soporte?

Resultado esperado

Las respuestas deben fundamentarse en la documentación técnica disponible.

Caso 3: Asistente Legal

Descripción

Un estudio jurídico desea construir un asistente que permita consultar normativa específica.

Corpus documental sugerido

- Ley de Protección de Datos Personales.
- Ley del Consumidor.
- Normativas laborales.
- Reglamentos internos.

Preguntas de ejemplo

- ¿Qué establece la Ley 21.719?
- ¿Cuáles son los derechos del consumidor?
- ¿Qué obligaciones tiene un empleador respecto a los datos personales?

Resultado esperado

El sistema debe citar correctamente los documentos utilizados para responder.

Caso 4: Asistente Médico

Descripción

Un centro médico requiere consultar protocolos clínicos y procedimientos internos.

Corpus documental sugerido

- Protocolos clínicos.
- Procedimientos médicos.
- Guías de atención.
- Manuales de operación.

Preguntas de ejemplo

- ¿Cuál es el protocolo para atención inicial?
- ¿Qué exámenes deben realizarse?
- ¿Cuáles son los pasos del procedimiento?

Resultado esperado

Las respuestas deben estar sustentadas por los documentos proporcionados.

5. Requisitos Técnicos Obligatorios

La solución deberá incluir obligatoriamente los siguientes componentes.

5.1 Ingesta Documental

El sistema debe cargar documentos desde archivos.

Formatos permitidos:

- PDF
 - TXT
 - Markdown (.md)
-

5.2 Fragmentación (Chunking)

Los documentos deberán dividirse en fragmentos adecuados para la recuperación semántica.

El equipo deberá justificar brevemente:

- Tamaño de chunk utilizado.
 - Solapamiento (overlap) utilizado.
-

5.3 Embeddings

La solución deberá generar embeddings para cada fragmento documental.

Tecnologías sugeridas:

- Ollama Embeddings
 - Nomic Embed
 - Sentence Transformers
-

5.4 Base Vectorial

Debe utilizarse una base vectorial para almacenar los embeddings.

Opciones sugeridas:

- ChromaDB
 - FAISS
-

5.5 Recuperación de Información

La consulta del usuario debe generar un embedding y recuperar los fragmentos más relevantes.

El sistema deberá mostrar al menos uno de los siguientes elementos:

- Fragmentos recuperados.
 - Fuente documental utilizada.
-

5.6 Generación de Respuesta

La respuesta deberá ser generada por un modelo de lenguaje utilizando:

Pregunta del usuario + Contexto recuperado

Opciones sugeridas, no mayor a **7B**:

- Llama 3
 - Gemma
 - Mistral
-

6. Funcionalidades Mínimas

La aplicación deberá permitir:

- ✓ Realizar preguntas.
 - ✓ Recuperar documentos relevantes.
 - ✓ Generar respuestas utilizando el contexto recuperado.
 - ✓ Mostrar las fuentes utilizadas.
 - ✓ Manejar preguntas cuya respuesta no exista en los documentos.
-

7. Demostración Obligatoria

Durante la presentación cada equipo deberá demostrar los siguientes escenarios.

Escenario 1: Consulta Simple

Ejemplo:

¿Cuál es la asistencia mínima requerida?

Escenario 2: Consulta Compleja

Ejemplo:

¿Qué requisitos debo cumplir para titularme y qué sucede si no los cumplo?

Escenario 3: Consulta Sin Respuesta Documental

Ejemplo:

Una pregunta cuya respuesta no exista dentro del corpus documental.

El sistema deberá reconocer la ausencia de información o indicar incertidumbre de manera razonable.

8. Entregables

Cada equipo deberá entregar lo siguiente.

Repositorio GitHub

Debe contener:

- Código fuente.
 - Instrucciones de ejecución.
 - Dependencias necesarias.
 - Archivos de configuración requeridos para ejecutar la solución.
-

Documento Breve (Máximo 3 Páginas)

Debe incluir:

Descripción del problema abordado.

Tecnologías utilizadas.

Flujo de funcionamiento de la solución.

Capturas de pantalla.

Dificultades encontradas.

Reflexión sobre ventajas y limitaciones de RAG. (No IA)

9. Presentación

Duración máxima:

7 minutos por equipo

Distribución sugerida:

- Problema abordado (1 minuto).
 - Arquitectura implementada (2 minutos).
 - Demostración funcional (3 minutos).
 - Reflexión y conclusiones (1 minuto).
 - Video grabado subido en canvas.
-

10. Rúbrica

Criterio	Puntaje
Carga e indexación documental	15
Generación de embeddings	15
Implementación de recuperación semántica	20
Integración con LLM	15
Calidad de las respuestas generadas	10
Presentación de fuentes recuperadas	10
Manejo de preguntas fuera del contexto documental	5
Calidad de la demostración	10
Total	100

Consideraciones Finales

Esta evaluación busca demostrar la comprensión práctica del paradigma RAG y su integración dentro de una arquitectura moderna de inteligencia artificial.

Se evaluará principalmente:

- Comprensión técnica.
- Correcta implementación.
- Capacidad de explicar el flujo de recuperación y generación de información.

No se evaluará el diseño visual avanzado ni funcionalidades adicionales fuera del alcance definido para este laboratorio.